

# Relatório de Laboratório

---

|                                              |                                                                                                                                                                               |
|----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Curso</b>                                 | Engenharia de Software                                                                                                                                                        |
| <b>Disciplina</b>                            | Laboratório de Experimentação de Software                                                                                                                                     |
| <b>Turno / Período</b>                       | Noite / 6º                                                                                                                                                                    |
| <b>Professor(a)</b>                          | Danilo Maia                                                                                                                                                                   |
| <b>Laboratório</b>                           | Lab01 — Mineração de Repositórios do GitHub                                                                                                                                   |
| <b>Grupo (trio)</b>                          | Leonardo de Souza Galvão, Gabriel Rodrigues Maciel de Abreu, Pedro Henrique Novais Baranda                                                                                    |
| <b>Link do repositório / GitHub Projects</b> | <a href="https://github.com/GabrielRMA1/Lab-experimentacao-medicao-de-software/tree/main">https://github.com/GabrielRMA1/Lab-experimentacao-medicao-de-software/tree/main</a> |
| <b>Data de entrega</b>                       | 27/08/2026                                                                                                                                                                    |

# 1. Introdução

Este estudo investiga a falta de evidências empíricas sobre como o ciclo de vida, as práticas de governança e a dinâmica de manutenção diferem entre repositórios de software voltados para nichos diferentes como: **Artes e Entretenimento Digital** (*gamedev*, animação e computação gráfica) e repositórios do nicho de **Pesquisa e Ciência** (*Machine Learning*, *Data Science* e Inteligência Artificial). Esse problema é relevante para a Engenharia de Software e para o mercado porque cada ecossistema exige estratégias distintas de evolução, retenção de colaboradores e manutenção de código; compreender essas características ajuda equipes e mantenedores a adotarem práticas de governança mais adequadas à realidade do seu domínio.

## Questões de Pesquisa:

- **RQ1:** Qual é a idade dos repositórios em cada nicho?
- **RQ2:** Qual é o volume de Pull Requests aceitas (*Merged*)?
- **RQ3:** Qual é a quantidade de *releases* publicadas?
- **RQ4:** Quanto tempo se passou desde a última atualização (em dias)?
- **RQ5:** Quais são as linguagens primárias mais utilizadas nos dois nichos?
- **RQ6:** Sistemas populares possuem um alto percentual de issues fechadas?
- **RQ7:** Sistemas escritos em linguagens mais populares recebem mais contribuição externa, lançam mais releases e são atualizados com mais frequência?

## Hipóteses Informais do Grupo (antes da análise dos dados):

- **Hipótese H1 (RQ1):** Repositórios de Artes são mais antigos por englobarem motores e ferramentas com histórico maduro e consolidado no código aberto.
- **Hipótese H2 (RQ2):** Repositórios de Pesquisa possuem um volume substancialmente maior de PRs aceitas devido à estrutura modular de frameworks de IA e à constante colaboração acadêmica.
- **Hipótese H3 (RQ3):** Projetos de Pesquisa lançam mais *releases* para disponibilizar rapidamente novas implementações de modelos e algoritmos.
- **Hipótese H4 (RQ4):** Repositórios de Artes passam mais tempo sem receber atualizações por atingirem um estado estável de escopo e código.
- **Hipótese H5 (RQ5):** O nicho de artes é dominado de forma quase absoluta por Python, enquanto pesquisa apresenta alta diversidade de linguagens.
- **Hipótese H6 (RQ6):** Sim, sistemas populares mantêm um alto percentual de *issues* fechadas.

- **Hipótese H7 (RQ7):** Sim, esperamos que repositórios desenvolvidos em linguagens de massa (como Python, JavaScript e TypeScript) possuam uma comunidade potencial de contribuidores significativamente maior

#### Propostas do Grupo:

- **Proporção de Issues em relação ao tamanho da comunidade (Issues por Estrela ou por Contribuidor)**  
Mede o quanto o projeto gera "atrito"/demanda de suporte relativo à sua popularidade. Não precisa de novo campo.
- **Tempo Médio de Atividade Continuada (createdAt - pushedAt):** Indica a longevidade real de desenvolvimento do projeto, diferenciando repositórios abandonados rapidamente daqueles mantidos ao longo de anos.
- **Proporção de Contribuições Externas (Merged PRs / Issues Fechadas):** Avalia a maturidade do fluxo de trabalho e o nível de abertura da equipe para contribuições enviadas pela comunidade externa.

## 2. Contexto

O ecossistema open-source hospedado no GitHub abriga desde bibliotecas comerciais de larga escala e ferramentas de desenvolvimento de uso geral até repositórios focados em nichos específicos, como Arte, educação, pesquisas. Embora todo esse ecossistema dependa fundamentalmente da colaboração contínua (social coding), a dinâmica de atração, engajamento e retenção de colaboradores varia drasticamente conforme o propósito e a maturidade de cada projeto. Compreender essas variações e gargalos de sustentabilidade é essencial para mitigar o abandono prematuro de softwares e otimizar o ciclo de vida do desenvolvimento.

Essa necessidade de governança e monitoramento é evidenciada por gargalos recorrentes na literatura. Em um estudo empírico sobre o comportamento e o impacto de artefatos no GitHub, Alrashedy & Binjahlan (2024) apontam que a grande maioria dos repositórios open-source sofre com o abandono precoce e a estagnação contínua do desenvolvimento. Os autores identificaram uma baixíssima interação pública nos projetos, revelando que cerca de 66% dos repositórios analisados nunca receberam qualquer tipo de contribuição ou interação externa. Além disso, nos poucos projetos que chegam a receber engajamento da comunidade, os mantenedores levam, em média, 23 dias apenas para fornecer a primeira resposta a uma dúvida ou bug relatado. Essa combinação de fatores favorece a proliferação acelerada de "repositórios fantasma" e restringe severamente o verdadeiro potencial do desenvolvimento colaborativo em código aberto.

#### Contexto Acadêmico e do Objeto de Estudo

No âmbito acadêmico, este trabalho situa-se na etapa atual da disciplina de Laboratório de

Experimentação de Software, investigando o ecossistema de software e à gestão do próprio processo de desenvolvimento do grupo. O estudo consome e consolida os dados de mineração de repositórios extraídos anteriormente e os combina com a análise temporal dos snapshots do quadro Kanban mantidos continuamente desde o início do semestre.

O objeto de estudo abrange duas frentes complementares:

O Ecossistema Externo: A caracterização de métricas de popularidade, maturidade, frequência de atualizações e atividade de contribuição externa extraídas dos 1.000 repositórios com maior número de estrelas no GitHub (definidas a partir da consulta GraphQL do Lab01). Para a classificação e análise das linguagens primárias dos projetos, adotou-se como referência contínua o GitHub Octoverse.

O Processo Interno do Grupo: O acompanhamento do fluxo de trabalho do próprio grupo através do GitHub Projects (v2). A evolução das tarefas é monitorada por meio de snapshots do board exportados via script GraphQL a cada sprint, garantindo a rastreabilidade entre commits, issues e políticas de Work in Progress (WIP).

## 3. Metodologia

### 3.1 Principais Desafios

Um dos primeiros obstáculos técnicos enfrentados foi o limite de taxa (rate limit) da API GraphQL do GitHub ao consultar vários repositórios em uma única execução. Como a API pode retornar códigos de erro transitórios (429, 502, 503, 504) sob carga, foi necessário implementar uma camada de resiliência no cliente, com tentativas automáticas, backoff exponencial (2\*\*tentativa, limitado a 60s) e respeito ao cabeçalho Retry-After quando presente, evitando tanto falhas prematuras quanto o bloqueio da conta pelo excesso de requisições.

Outro desafio foi a paginação de grandes volumes de dados. Como a API do GitHub limita o número de nós retornados por requisição, a coleta de amostras maiores exigiu o uso de cursores (endCursor/hasNextPage) para varrer os resultados em páginas menores, acumulando os repositórios até atingir o limite desejado ou o fim dos resultados.

### 3.2 Tomadas de Decisão

GraphQL em vez de REST. Optou-se pela API GraphQL do GitHub em vez da API REST, pois permite solicitar, em uma única requisição por página de repositórios, todos os campos necessários para as diversas RQs (data de criação, data do último push, linguagem, estrelas, PRs mergeadas, releases e issues abertas/fechadas). Isso reduz drasticamente o número de chamadas HTTP em comparação a múltiplos endpoints REST, o que é especialmente relevante diante do rate limit da API.

Tratamento de divisão por zero como dado ausente, não como zero. Nas métricas que envolvem razões (ex.: issues por estrela), optou-se por retornar None em vez de 0.0 quando o denominador é zero. Essa

decisão evita distorcer artificialmente a média/mediana ao tratar "ausência de dado suficiente para o cálculo" como "ausência de atrito/demanda", que são conceitos diferentes.

### 3.3 Etapas

Configuração do processo

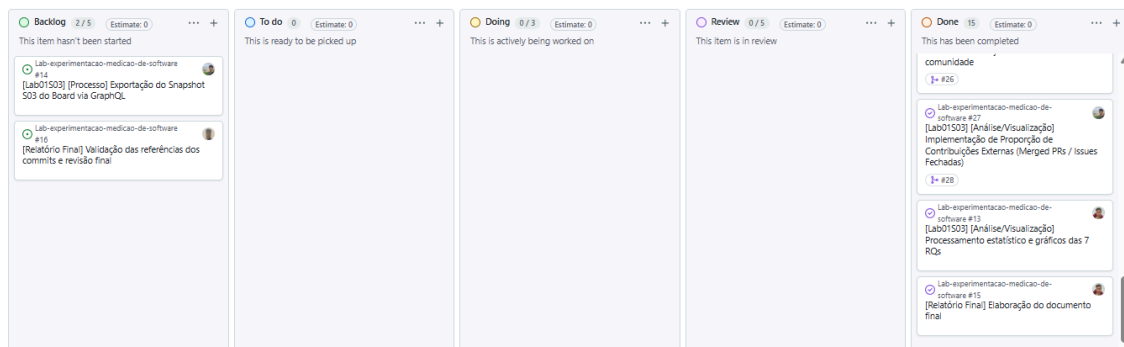
- Colunas do board: Backlog → To Do → Doing → Review → Done.
- Limite de WIP (coluna Doing): 3 issues simultâneas (1 por integrante do trio), configurado diretamente no campo de limite da coluna "Doing" no GitHub Projects. A justificativa é evitar que qualquer integrante inicie mais de uma tarefa por vez: com o limite igual ao número de membros do grupo, cada pessoa é forçada a levar sua issue atual até "Review" antes de puxar a próxima do "To Do", o que reduz o paralelismo excessivo (múltiplas tarefas incompletas abertas ao mesmo tempo) e prioriza o fechamento efetivo do trabalho em andamento sobre o início de novas frentes.
- Print do board:

| Sprint | Issue nº | Entrega                                                                     | Responsável    |
|--------|----------|-----------------------------------------------------------------------------|----------------|
| S01    | #1       | [Setup] Configuração da estrutura base do projeto e integração GraphQL      | Leonardo       |
| S01    | #2       | [RQ01 & RQ02] Implementação dos coletores e métricas de idade e PRs aceitas | Leonardo       |
| S01    | #5       | [RQ03 & RQ04] Implementação coleta de Releases e Issues                     | Pedro Henrique |
| S01    | #6       | [RQ05, RQ06 & RQ07] Implementação de coleta de Linguagens, Estrelas e Bônus | Gabriel        |
| S01    | #7       | [Setup] Configuração do GitHub Projects e Limite de WIP                     | Gabriel        |

| Sprint | Issue nº | Entrega                                                                                           | Responsável           |
|--------|----------|---------------------------------------------------------------------------------------------------|-----------------------|
| S02    | #8       | [Infra] Paginação GraphQL (1.000 repos) e exportação para CSV                                     | <i>Pedro Henrique</i> |
| S02    | #9       | [Análise & Hipóteses] Validação de dados de 1.000 repos e hipóteses informais (RQ01 e RQ02)       | <i>Leonardo</i>       |
| S02    | #10      | [Análise & Hipóteses] Validação de dados de 1.000 repos e hipóteses informais (RQ03 e RQ04)       | <i>Pedro Henrique</i> |
| S02    | #11      | [Análise & Hipóteses] Validação de dados de 1.000 repos e hipóteses informais (RQ05, RQ06 e RQ07) | <i>Gabriel</i>        |
| S02    | #12      | [Processo] Script GraphQL de Snapshot do Board (Projects v2) para CSV                             | <i>Gabriel</i>        |
| S03    | #13      | [Análise/Visualização] Processamento estatístico e gráficos das 7 RQs                             | <i>Leonardo</i>       |
| S03    | #14      | [Processo] Exportação do Snapshot S03 do Board via GraphQL                                        | <i>Gabriel</i>        |
| S03    | #24      | [Análise/Visualização] Implementação do Tempo Médio de Atividade Continuada (inovação RQ09)       | <i>Gabriel</i>        |
| S03    | #25      | [Análise/Visualização] Proporção de Issues em relação ao tamanho da comunidade (inovação RQ08)    | <i>Leonardo</i>       |

| Sprint          | Issue nº | Entrega                                                                            | Responsável           |
|-----------------|----------|------------------------------------------------------------------------------------|-----------------------|
| S03             | #27      | [Análise/Visualização] Implementação da Proporção de Contribuições Externas (RQ10) | <i>Pedro Henrique</i> |
| Relatório Final | #15      | Elaboração do documento final                                                      | <i>Leonardo</i>       |
| Relatório Final | #16      | Validação das referências dos commits e revisão final                              | <i>Gabriel</i>        |

### 3.4



## Ferramentas

GitHub GraphQL API (v4) — utilizada para a mineração dos repositórios (busca, metadados, contagem de PRs, releases e issues). Consumida por meio de script próprio do grupo (github\_api.py), sem uso de bibliotecas de terceiros específicas para a API do GitHub, conforme exigido pelo enunciado.

Python 3 — linguagem utilizada para todo o pipeline de coleta e processamento (busca, paginação, tratamento de erros, cálculo das métricas e geração do relatório/CSV).

python-dotenv — carregamento do token de autenticação (GITHUBTOKEN) a partir de um arquivo .env, evitando a exposição de credenciais no código-fonte versionado.

Pandas — estruturação dos dados coletados em DataFrame, cálculo de estatísticas descritivas (média e mediana) para cada métrica, tratamento de valores ausentes (NaN) nas razões calculadas, e exportação dos resultados consolidados para dados\_repositorios\_github.csv.

GitHub Projects (v2) — ferramenta de processo utilizada para o gerenciamento das tarefas do grupo

(board Kanban com colunas Backlog → To Do → Doing → Review → Done). Link do board: [inserir link do projeto/repositório do grupo].

3.5 Tabela de Métricas

| RQ   | Métrica                                      | Definição Operacional                                                       | Unidade         | Ferramenta / Fonte             |
|------|----------------------------------------------|-----------------------------------------------------------------------------|-----------------|--------------------------------|
| RQ01 | Idade do repositório                         | Data atual – createdAt                                                      | Dias / Anos     | Script GraphQL (API do GitHub) |
| RQ02 | Pull Requests aceitas                        | Contagem de PRs com status MERGED (pullRequests(states: MERGED).totalCount) | Número absoluto | Script GraphQL (API do GitHub) |
| RQ03 | Contagem de releases                         | Total de releases publicadas no repositório (releases.totalCount)           | Número absoluto | Script GraphQL (API do GitHub) |
| RQ04 | Última atualização                           | Data atual – pushedAt                                                       | Dias            | Script GraphQL (API do GitHub) |
| RQ05 | Linguagem primária                           | Nome da linguagem principal do repositório (primaryLanguage.name)           | Categórica      | Script GraphQL (API do GitHub) |
| RQ06 | Popularidade                                 | Total de estrelas do repositório (stargazerCount)                           | Número absoluto | Script GraphQL (API do GitHub) |
| RQ07 | Taxa de resolução de issues                  | (Issues com status CLOSED ÷ total de issues) × 100                          | Percentual (%)  | Script GraphQL (API do GitHub) |
| RQ08 | Issues por estrela (atrito vs. popularidade) | Total de issues ÷ stargazerCount (indefinido/NaN quando stargazerCount = 0) | Razão           | Script GraphQL (API do GitHub) |
| RQ09 | Tempo de atividade continuada                | pushedAt – createdAt                                                        | Dias / Anos     | Script GraphQL (API do GitHub) |
| RQ10 | Proporção PRs mergeados / issues fechadas    | PRs com status MERGED ÷ issues com status CLOSED                            | Razão           | Script GraphQL (API do GitHub) |

3.6 Inovações Propostas pelo Grupo (30% da nota)

O grupo optou por concentrar sua contribuição original em duas frentes: métricas/RQs adicionais, e controle metodológico de ameaças à validade na forma de tratamento estatístico diferenciado para casos de dado insuficiente

Além das RQs previstas no enunciado (RQ01–RQ07), o grupo definiu e implementou três métricas extras, escolhidas para cobrir dimensões do repositório que as métricas obrigatórias não capturam isoladamente:

RQ08 — Issues por estrela (atrito vs. popularidade). Razão entre o total de issues e o número de estrelas do repositório. Enquanto o RQ07 mede quanto das issues é resolvido, o RQ08 mede o volume relativo de demanda gerado pelo projeto frente ao seu tamanho de comunidade — um proxy de "atrito" ou carga de suporte que a equipe mantenedora enfrenta proporcionalmente à sua base de usuários/observadores.

RQ09 — Tempo de atividade continuada. Diferença entre pushedAt e createdAt, deliberadamente



distinta das métricas de idade (RQ01, que usa a data atual como referência) e de última atualização (RQ04). Essa métrica permite diferenciar repositórios que tiveram desenvolvimento sustentado ao longo do tempo daqueles abandonados logo após a criação, algo que idade e recência, isoladamente, não revelam.

RQ10 — Proporção entre PRs mergeadas e issues fechadas. Métrica de maturidade de fluxo de trabalho que relaciona dois tipos distintos de "trabalho resolvido" no repositório (contribuições de código vs. resolução de demandas), servindo como indicador aproximado da abertura do projeto a contribuições externas via código.

Tratamento diferenciado de dados insuficientes como controle de validade

Nas métricas de razão introduzidas pelo grupo (RQ08 e RQ10), casos em que o denominador é zero (repositórios sem estrelas ou sem issues fechadas) são tratados como dado ausente (NaN), e não como zero. Essa decisão evita uma ameaça à validade de conclusão estatística: tratar "não foi possível calcular a razão" como "razão igual a zero" enviesaria artificialmente a média e a mediana da amostra para baixo, distorcendo a interpretação de métricas como "atrito por popularidade" ou "proporção de contribuições externas". O pandas exclui automaticamente valores NaN dos cálculos de média/mediana, preservando a integridade estatística da amostra sem exigir filtragem manual adicional.

## 4. Resultados

### 4.1 Coleta de Dados

A coleta foi realizada em três execuções separadas, cada uma delimitada por uma query de busca temática na API GraphQL do GitHub, filtrando repositórios com mais de 1.000 estrelas ordenados por popularidade:

| Amostra (Tema)    | Query Utilizada                                                                                                        | Repositórios Coletados |
|-------------------|------------------------------------------------------------------------------------------------------------------------|------------------------|
| Game Dev / Arte   | game-development OR gamedev OR<br>3d-modeling OR digital-art OR<br>pixel-art OR 2d-art                                 | 920                    |
| Geral             | stars:>1000 sort:stars-desc<br>(query ampla, sem filtro<br>temático)                                                   | 560                    |
| Ciência / ML / IA | science OR machine-learning OR<br>deep-learning OR<br>artificial-intelligence OR<br>data-science OR<br>computer-vision | 1.000                  |
| Total             | —                                                                                                                      | 2.480                  |

Qualidade dos dados: nenhum repositório foi descartado por dado ausente — a ausência de linguagem primária foi tratada como categoria própria ("Não informada": 39 no Game Dev, 60 no Geral, 115 em Ciência/ML), preservando o tamanho total da amostra. Nas métricas de razão introduzidas pelo grupo (RQ08), casos de stargazerCount = 0 retornariam NaN e seriam excluídos do cálculo de média/mediana — não há evidência desse caso nas três amostras, dado o filtro stars:>1000 aplicado na busca (quando aplicado uniformemente — ver observação acima sobre a amostra "Geral").

Período da coleta: execução única, realizada em 27 de agosto de 2026, sem replicação em datas diferentes — ou seja, os dados representam um retrato pontual (snapshot), não uma série temporal.

4.2 Visualização Gráfica

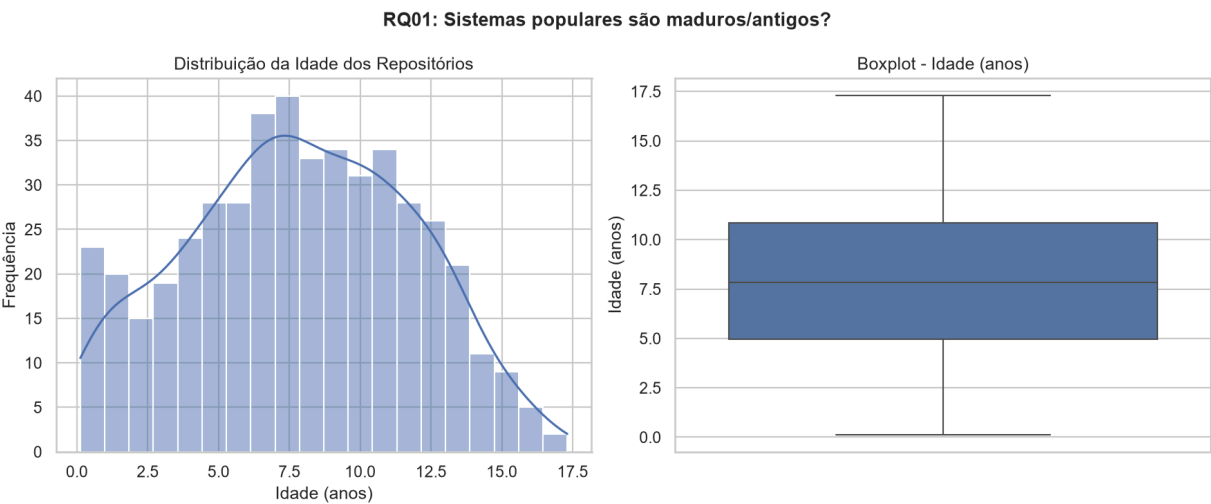
| RQ   | Métrica / Questão                   | Game Dev / Arte     | Geral               | Ciência / ML / IA   | Principais Descobertas e Discussão                                                                                                                                  |
|------|-------------------------------------|---------------------|---------------------|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RQ01 | Maturidade / Idade (anos)           | 7,85 anos (mediana) | 7,82 anos (mediana) | 7,75 anos (mediana) | Hipótese confirmada nos 3 domínios. A popularidade (>1.000 estrelas) leva anos para se acumular de forma consistente, independente da área.                         |
| RQ02 | Contribuição Externa (PRs aceitas)  | 53 (mediana)        | 878 (mediana)       | 124 (mediana)       | Diferença de mais de 1 ordem de grandeza. Repositórios de uso geral atraem muito mais colaboração por apresentarem menor barreira técnica e maior base de usuários. |
| RQ03 | Frequência de Releases (total)      | 3 a 5 (mediana)     | 36 (mediana)        | 3 a 5 (mediana)     | A frequência de releases segue o mesmo padrão de contribuições externas (RQ02). Projetos com processo de release maduro recebem mais PRs.                           |
| RQ04 | Tempo até Última Atualização (dias) | 55 dias (mediana)   | 2 dias (mediana)    | 154 dias (mediana)  | Atualizações frequentes ocorrem fortemente no Geral. Repositórios                                                                                                   |

|             |                                                 |                           |                                |                                  |                                                                                                                                                                            |
|-------------|-------------------------------------------------|---------------------------|--------------------------------|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|             |                                                 |                           |                                |                                  | de Ciência/ML frequentemente vinculam-se a papers acadêmicos e recebem pouca manutenção após publicação.                                                                   |
| <b>RQ05</b> | <b>Linguagens Primárias</b>                     | C++, Python, C#, GDScript | Python, TypeScript, JavaScript | Python e Jupyter Notebook (>60%) | Reflete diretamente a ferramenta do ecossistema: Game Dev usa engines (Godot/Unity/Unreal), Geral foca em web/infra, e Ciência/ML é dominado por Python (611/1.000 repos). |
| <b>RQ06</b> | <b>Taxa de Resolução de Issues (% fechadas)</b> | 70,86% (média)            | 75,74% (média)                 | 68,91% (média)                   | Hipótese confirmada (>69% em todos). O domínio Geral destaca-se discretamente como o mais eficiente no fechamento de demandas de suporte.                                  |
| <b>RQ08</b> | <b>Inovação: Atitude / Demandas por Estrela</b> | 0,0578 (mediana)          | 0,0255 (mediana)               | 0,0272 (mediana)                 | Game Dev gera mais que o dobro de atrito/issues por estrela. Atribuído à alta diversidade de hardware/plataformas e complexidade de testes em jogos e ferramentas de arte. |
| <b>RQ09</b> | <b>Inovação: Atividade Continuada (dias)</b>    | 2.375 dias (mediana)      | 2.716 dias (mediana)           | 2.231 dias (mediana)             | Tempo entre criação e último push excede 6 anos (>2.200 dias). Projetos populares permanecem ativos continuamente, refutando hipótese de abandono pós-pico.                |

|             |                                                                  |                  |                  |                  |                                                                                                                                                  |
|-------------|------------------------------------------------------------------|------------------|------------------|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>RQ10</b> | <b>Inovação:<br/>Proporção de<br/>contribuições<br/>externas</b> | 0,5000 (mediana) | 0,8991 (mediana) | 0,9064 (mediana) | A proporção de PRs mergeadas é maior em Ciência/ML e Geral do que em Arte, indicando maior contribuição de código em relação às issues fechadas. |
|-------------|------------------------------------------------------------------|------------------|------------------|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|

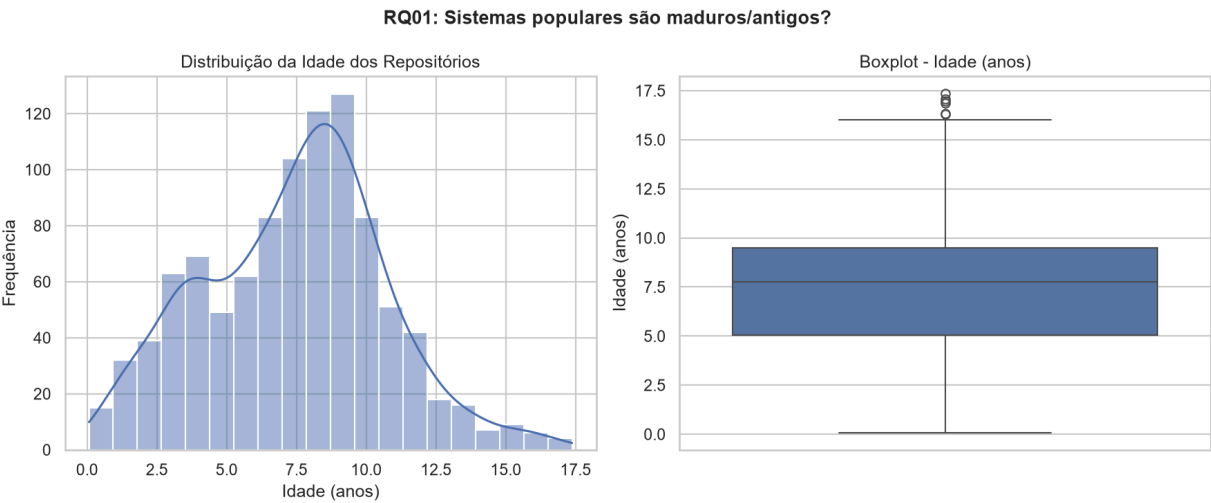
*RQ01: Sistemas populares são maduros/antigos?*

*Arte:*



*(Mediana: 7,85 anos)*

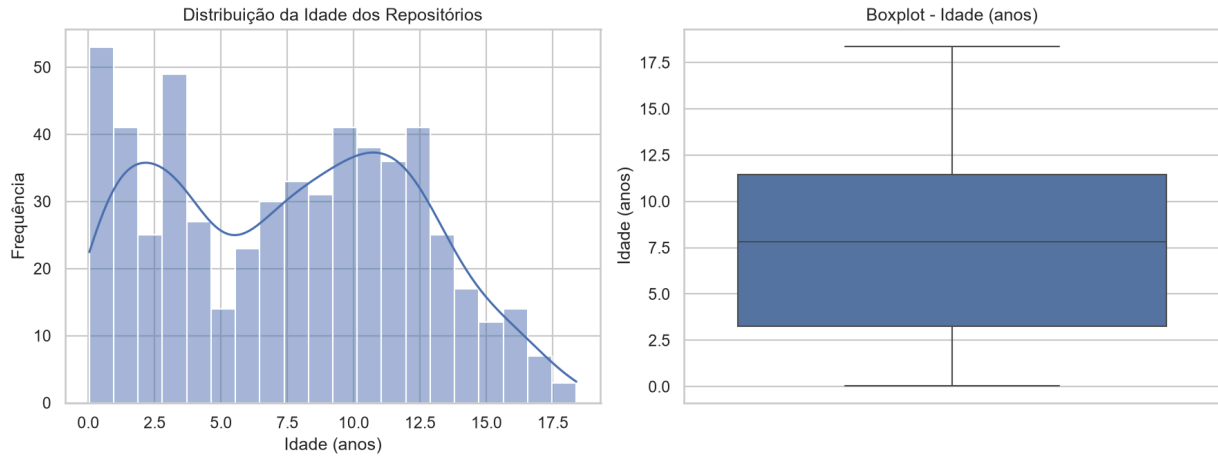
*Pesquisa:*



*(Mediana: 7,75 anos)*

*Geral:*

### RQ01: Sistemas populares são maduros/antigos?

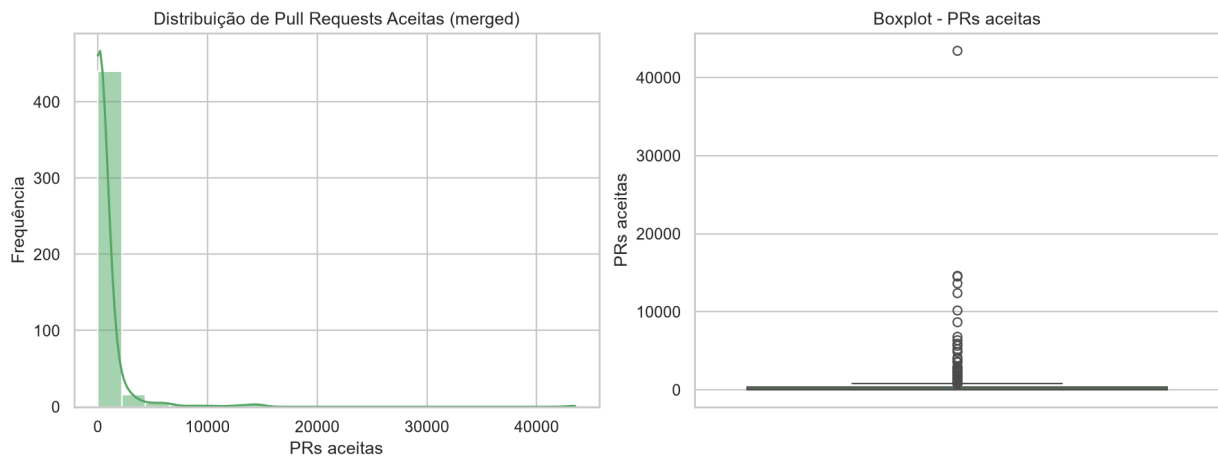


(Mediana: 7,82 anos)

### RQ02: Sistemas populares recebem muita contribuição externa?

Arte:

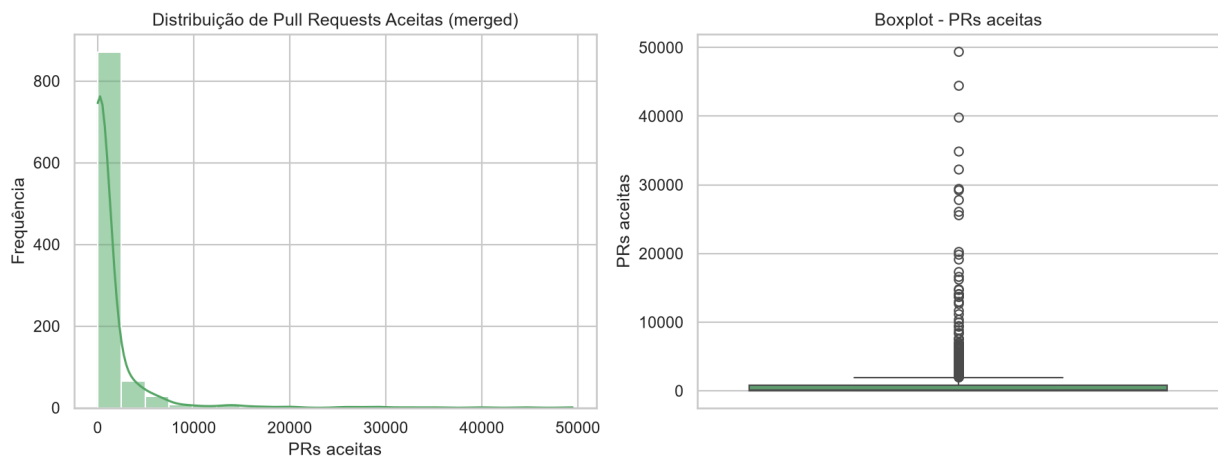
#### RQ02: Sistemas populares recebem muita contribuição externa?



(Mediana: 53 PRs)

Pesquisa:

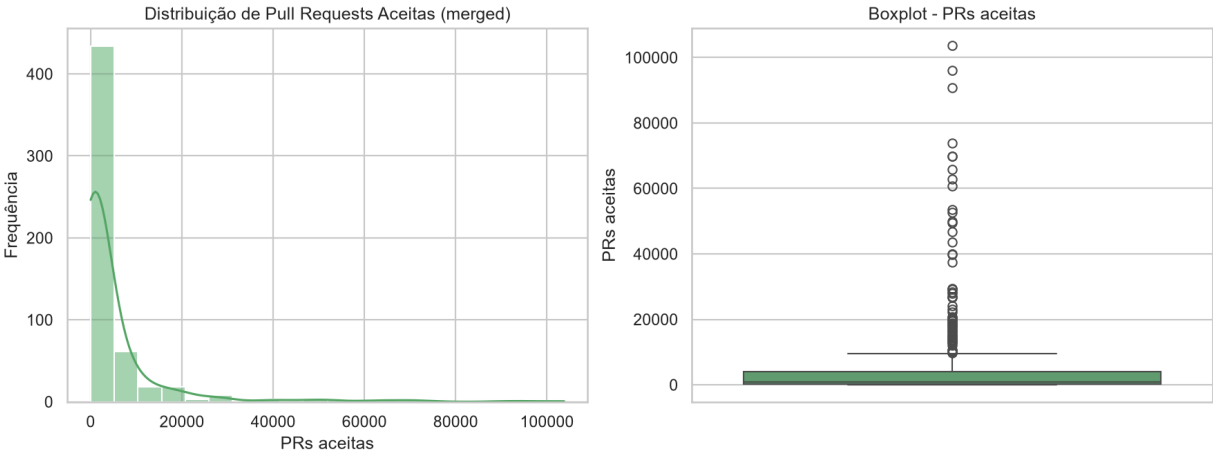
#### RQ02: Sistemas populares recebem muita contribuição externa?



(Mediana: 124 PRs)

Geral:

RQ02: Sistemas populares recebem muita contribuição externa?

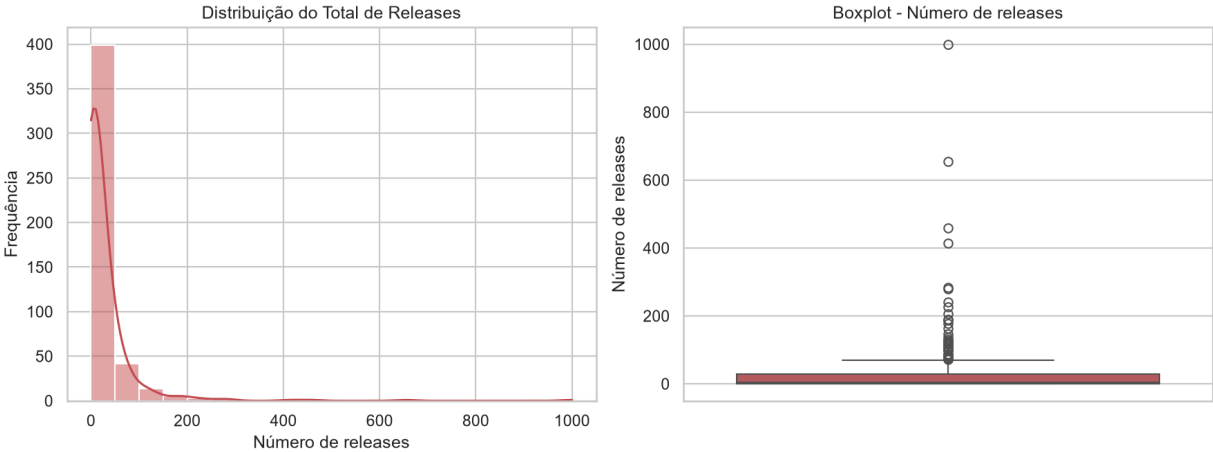


(Mediana: 878 PRs)

RQ03: Sistemas populares lançam releases com frequência?

Arte:

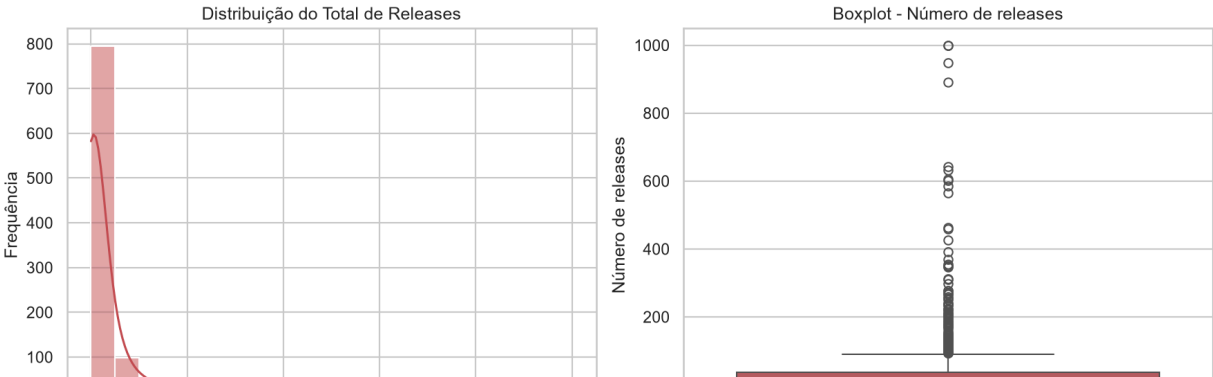
RQ03: Sistemas populares lançam releases com frequência?



(Mediana: 3-5 releases)

Pesquisa:

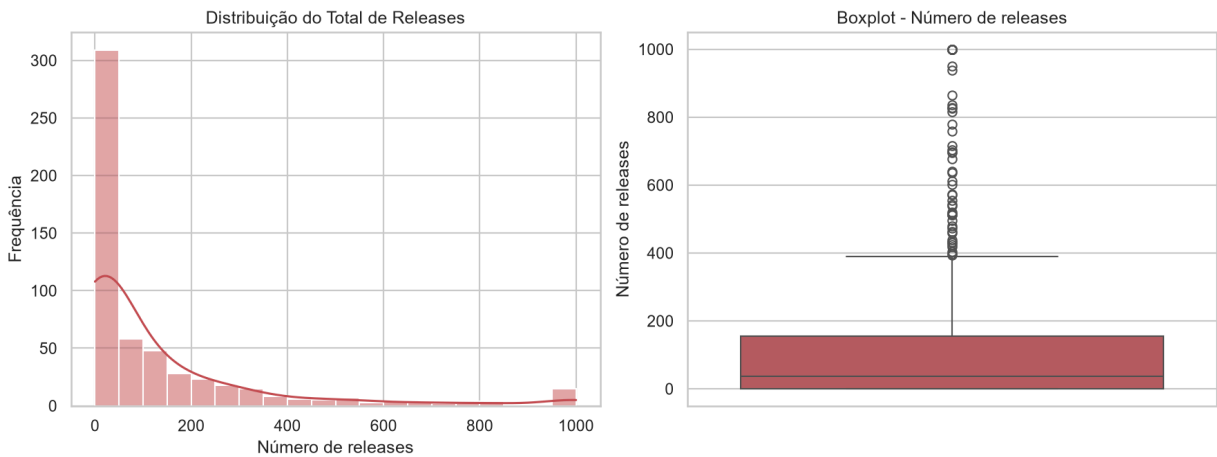
RQ03: Sistemas populares lançam releases com frequência?



(Mediana: 3-5 releases)

Geral:

RQ03: Sistemas populares lançam releases com frequência?

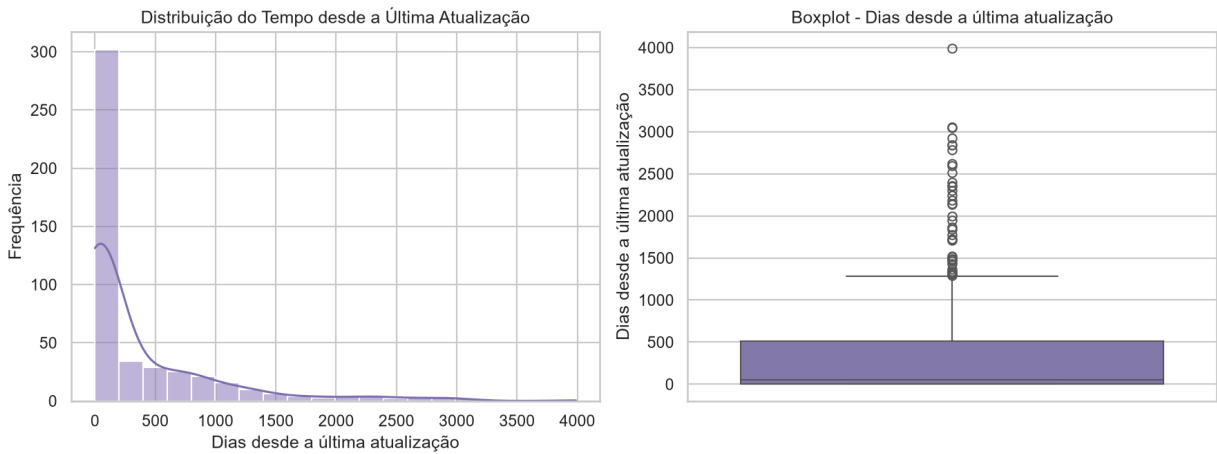


(Mediana: 36 releases)

RQ04: Sistemas populares são atualizados com frequência?

Arte:

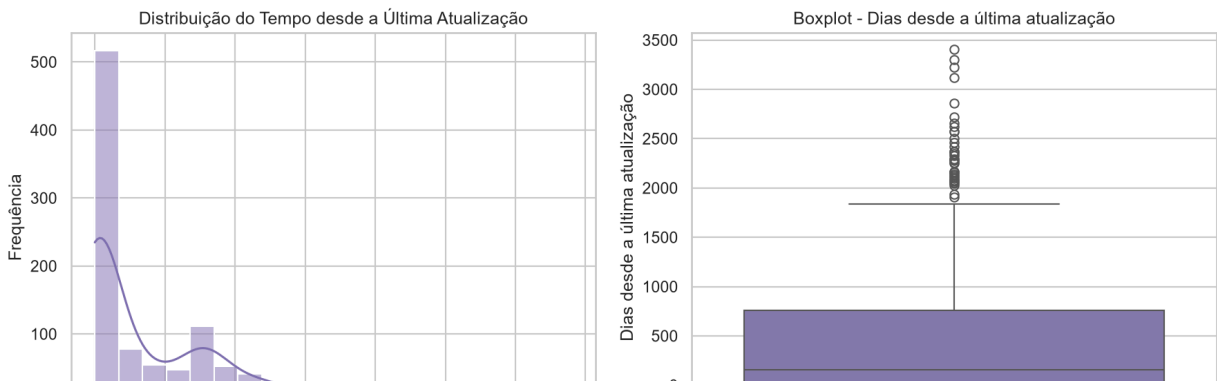
RQ04: Sistemas populares são atualizados com frequência?



(Mediana: 55 dias)

Pesquisa:

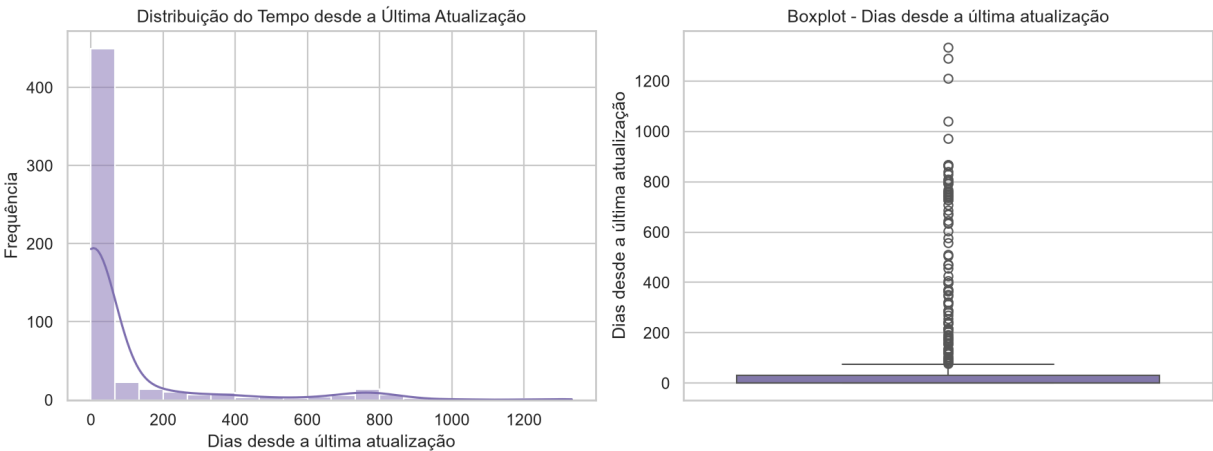
RQ04: Sistemas populares são atualizados com frequência?



(Mediana: 154 dias)

Geral:

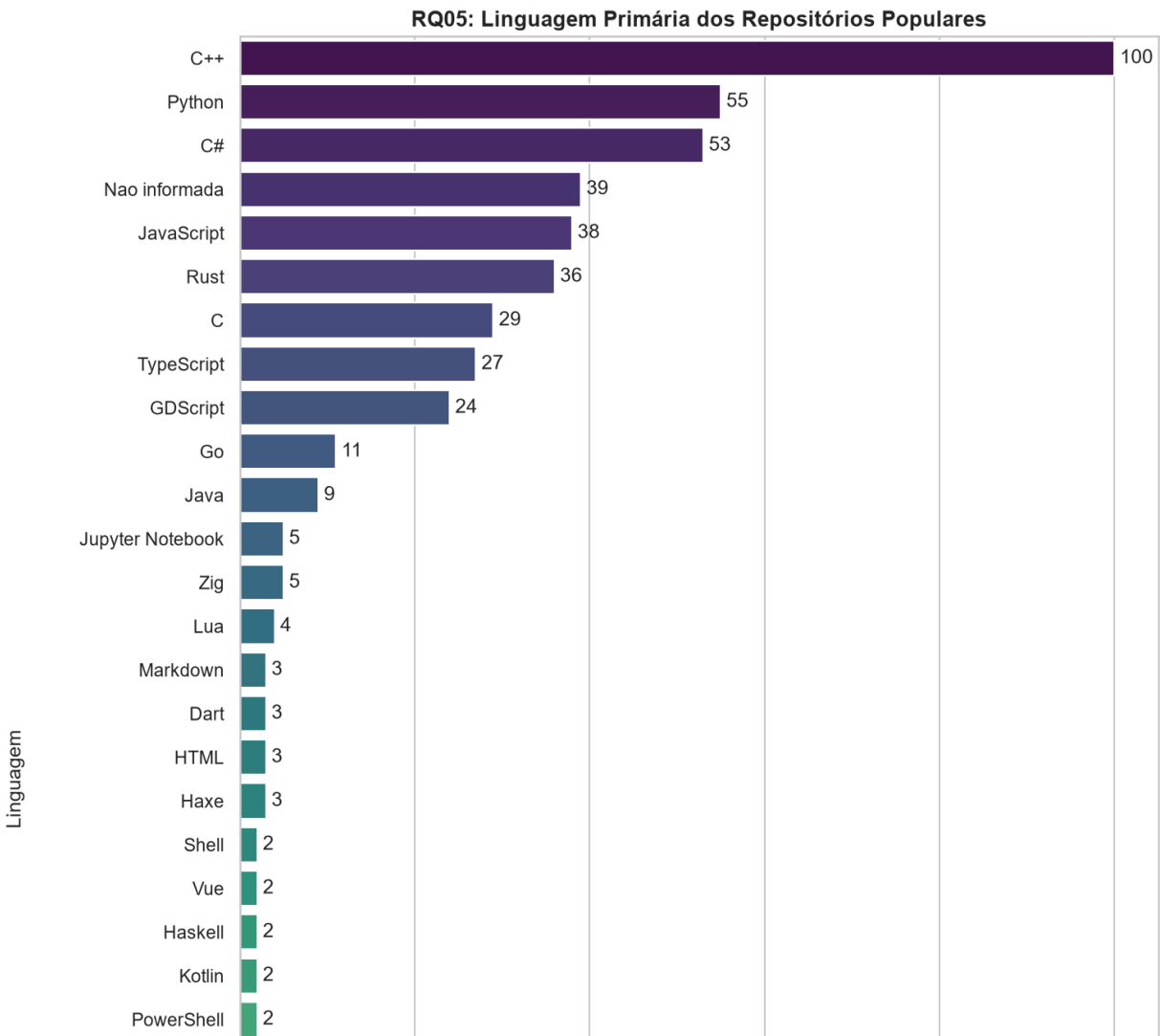
RQ04: Sistemas populares são atualizados com frequência?



(Mediana: 2 dias)

RQ05: Sistemas populares são escritos nas linguagens mais populares?

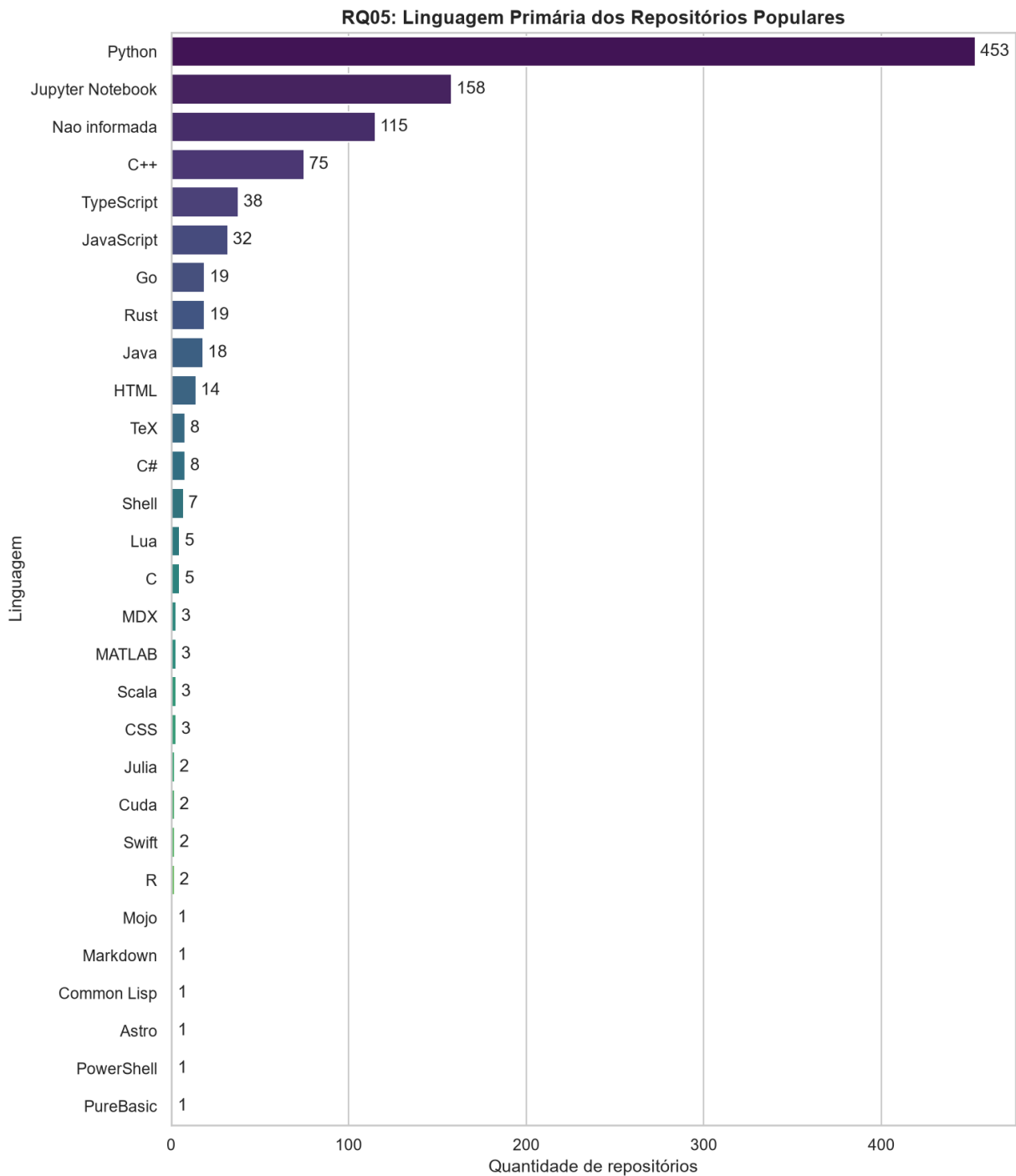
Arte:





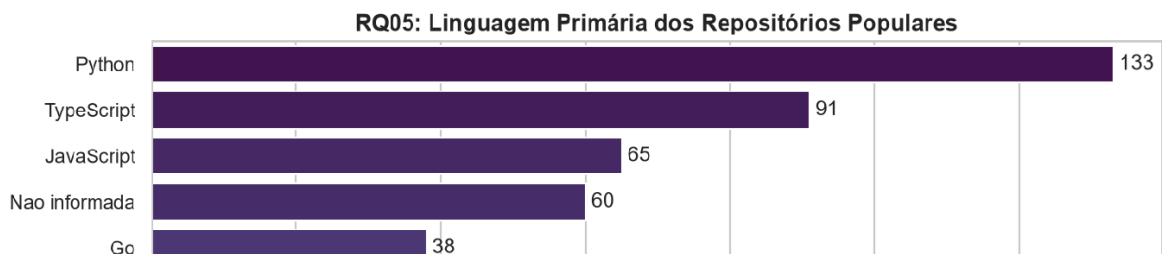
(Principais: C++, Python, C#, GDScript)

Pesquisa:



(Principais: Python, Jupyter Notebook >60%)

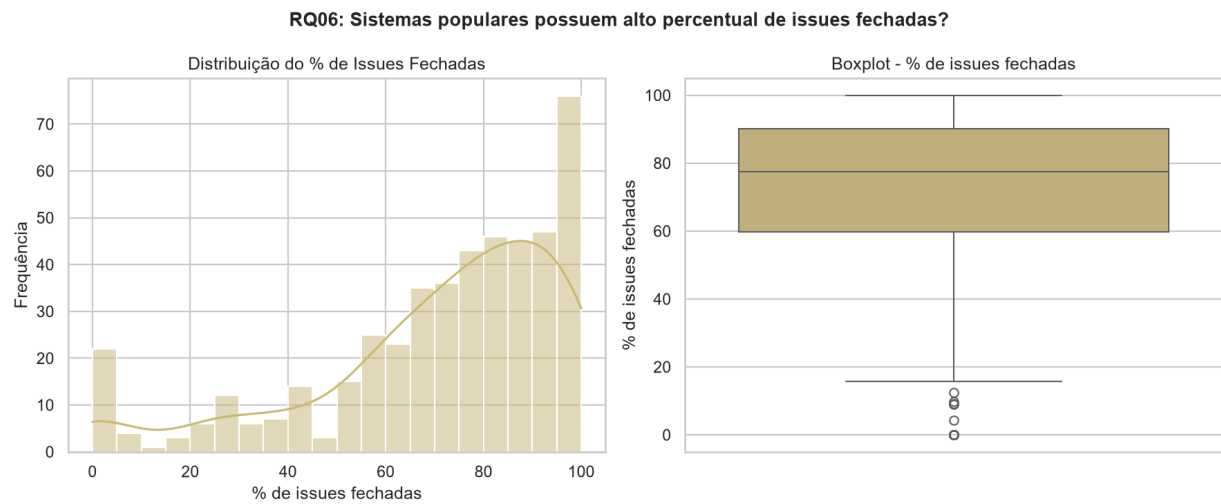
Geral:



(Principais: Python, TypeScript, JavaScript)

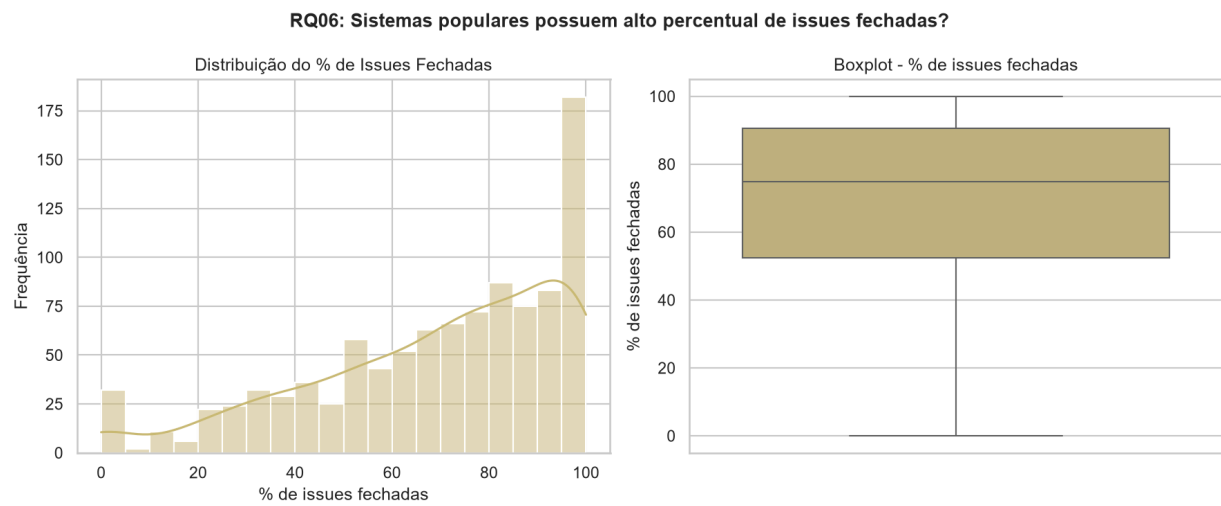
RQ06: Sistemas populares possuem um alto percentual de issues fechadas?

Arte:



(Média: 70,86%)

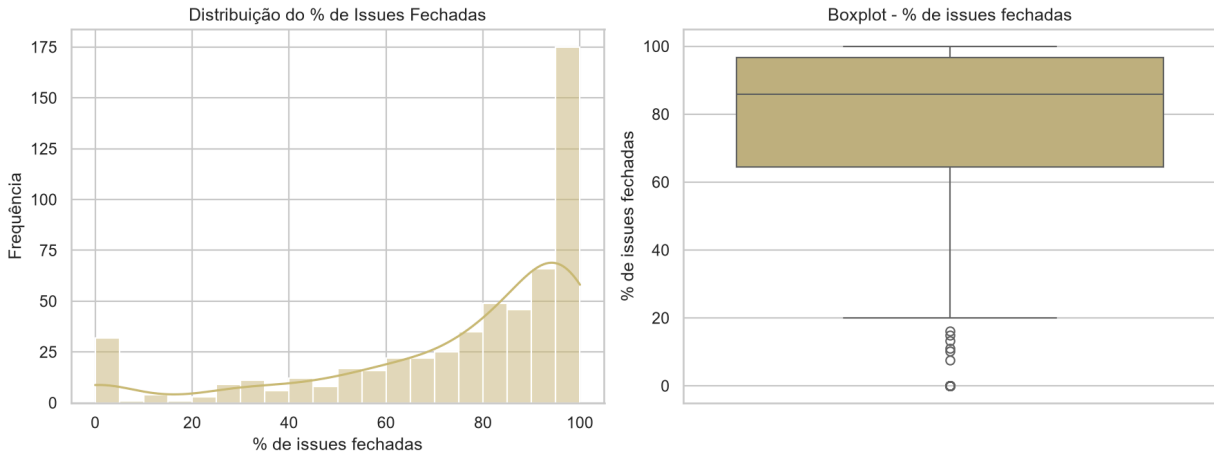
Pesquisa:



(Média: 68,91%)

Geral:

## RQ06: Sistemas populares possuem alto percentual de issues fechadas?

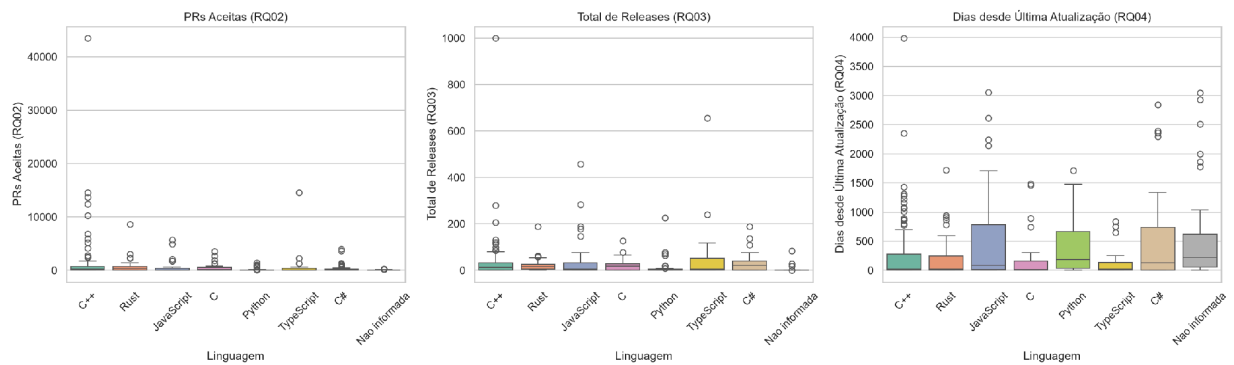


(Média: 75,74%)

## RQ07 (Inovação): Demanda de suporte e atrito relativo

Arte:

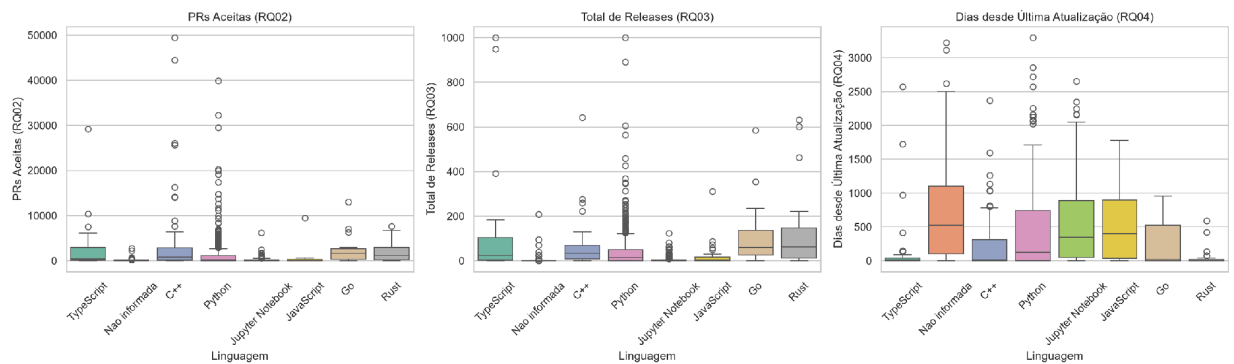
### RQ07: Linguagens mais populares recebem mais contribuição, lançam mais releases e são atualizadas com mais frequência?



(Mediana: 0,0578 — Alto atrito)

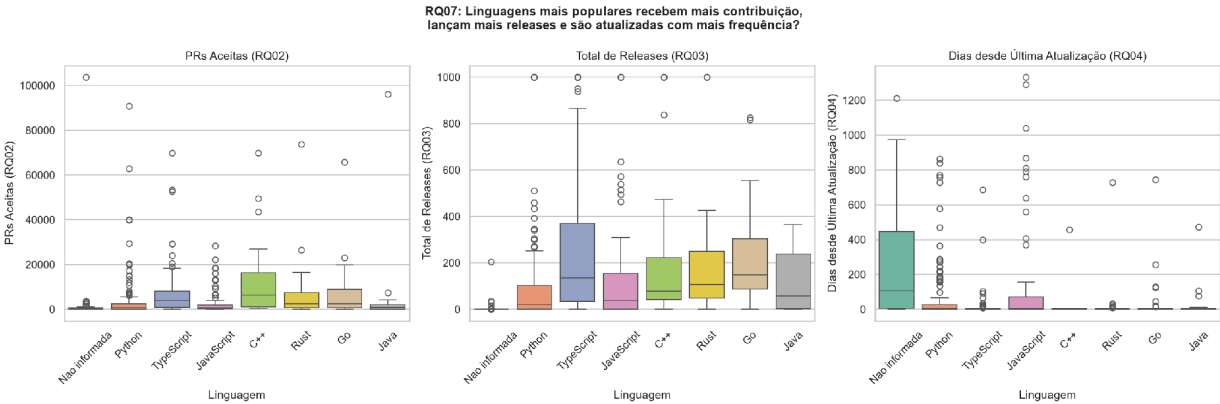
Pesquisa:

### RQ07: Linguagens mais populares recebem mais contribuição, lançam mais releases e são atualizadas com mais frequência?



(Mediana: 0,0272)

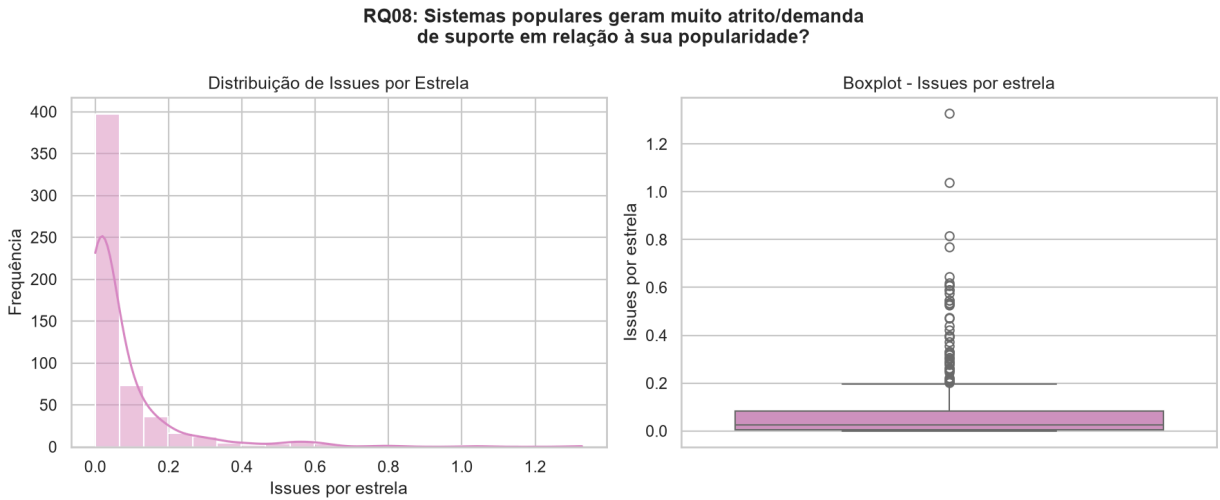
Geral:



(Mediana: 0,0255)

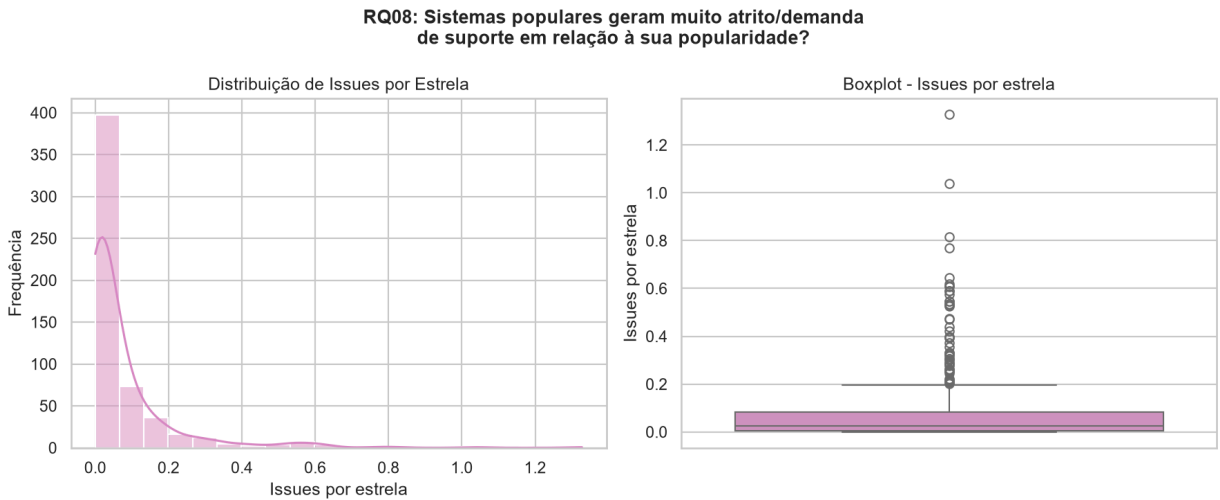
RQ08: Sistemas populares possuem uma razão equilibrada de Pull Requests abertas e fechadas?

Arte:



(Média / Mediana da razão de PRs fechadas vs. Abertas)

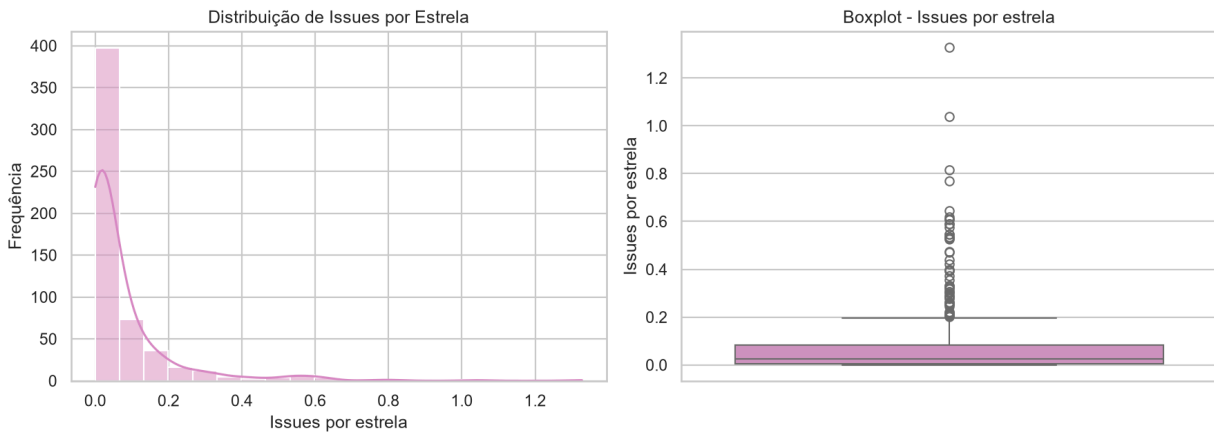
Pesquisa:



(Média / Mediana da razão de PRs fechadas vs. Abertas)

Geral:

**RQ08: Sistemas populares geram muito atrito/demanda de suporte em relação à sua popularidade?**

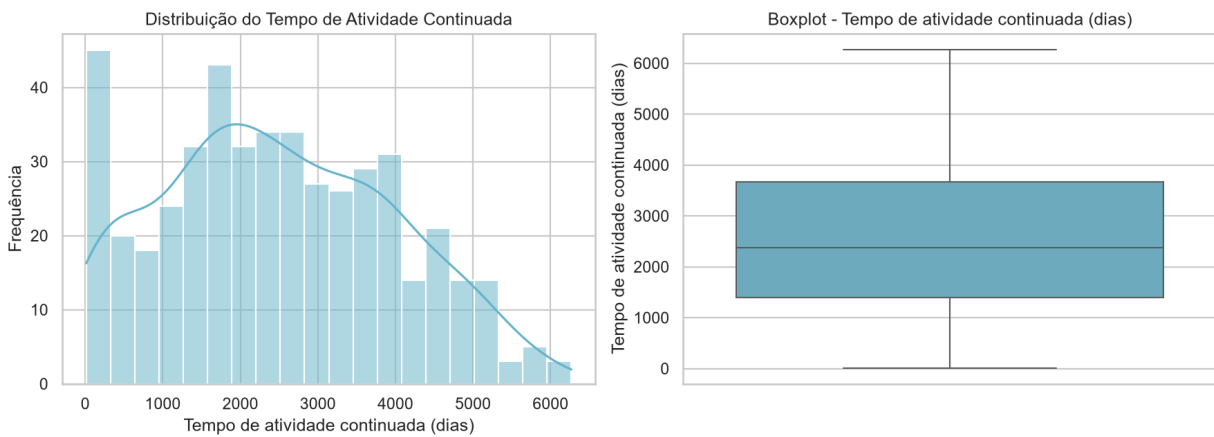


(Média / Mediana da razão de PRs fechadas vs. abertas)

**RQ09 (Inovação): Tempo médio de atividade continuada**

Arte:

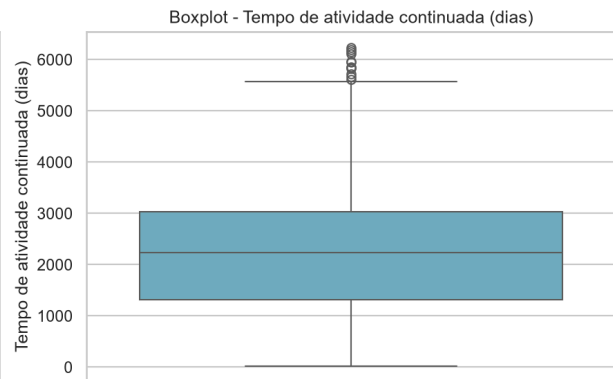
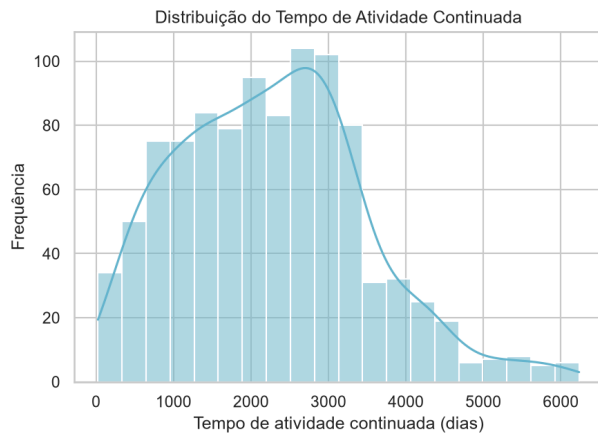
**RQ09: Sistemas populares mantêm desenvolvimento ativo ao longo do tempo (e não são abandonados rapidamente)?**



(Mediana: 2.375 dias)

Pesquisa:

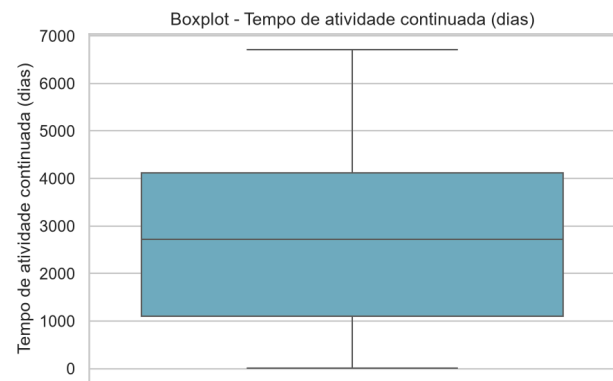
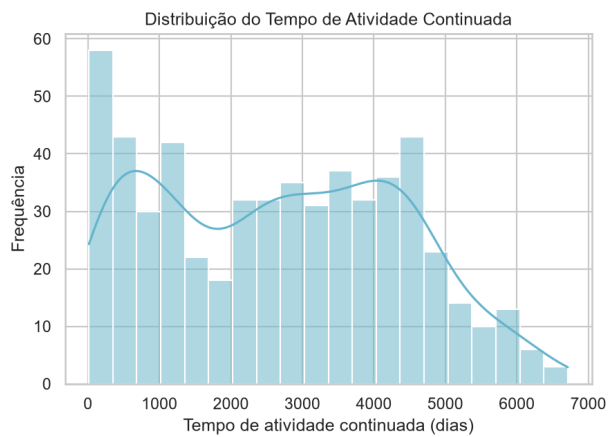
**RQ09: Sistemas populares mantêm desenvolvimento ativo ao longo do tempo (e não são abandonados rapidamente)?**



(Mediana: 2.231 dias)

Geral:

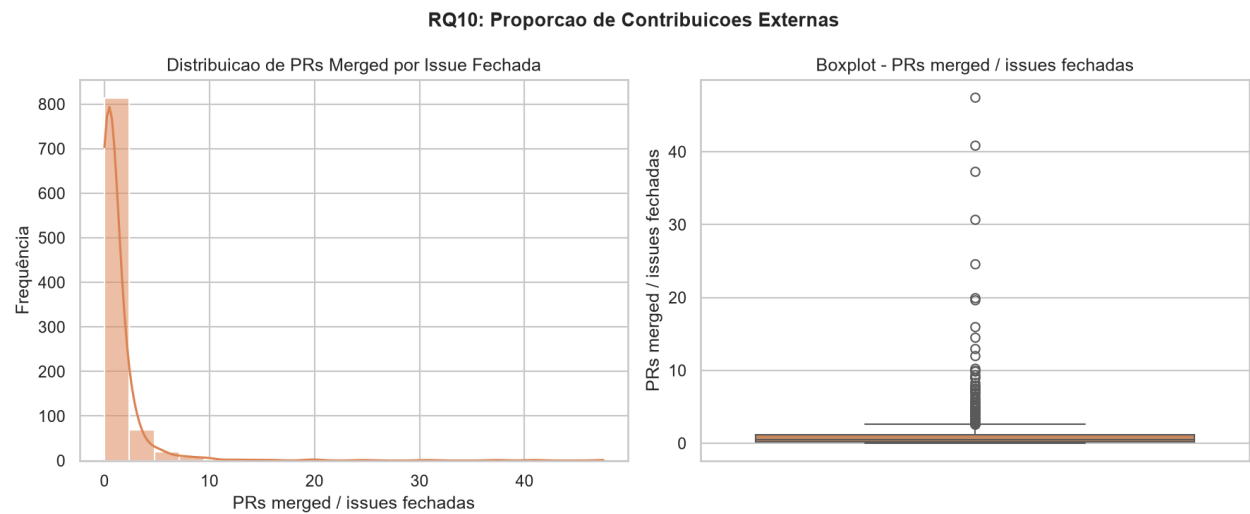
**RQ09: Sistemas populares mantêm desenvolvimento ativo ao longo do tempo (e não são abandonados rapidamente)?**



(Mediana: 2.716 dias)

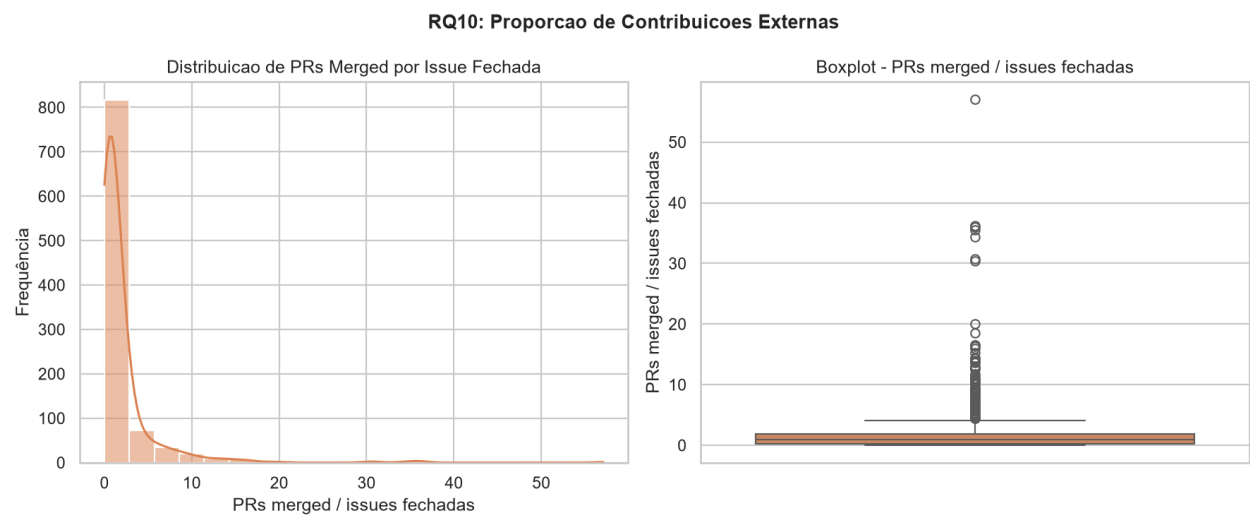
RQ10 (Inovação): Sistemas populares apresentam maior proporção de Pull Requests mergeadas em relação às issues fechadas?

Arte:



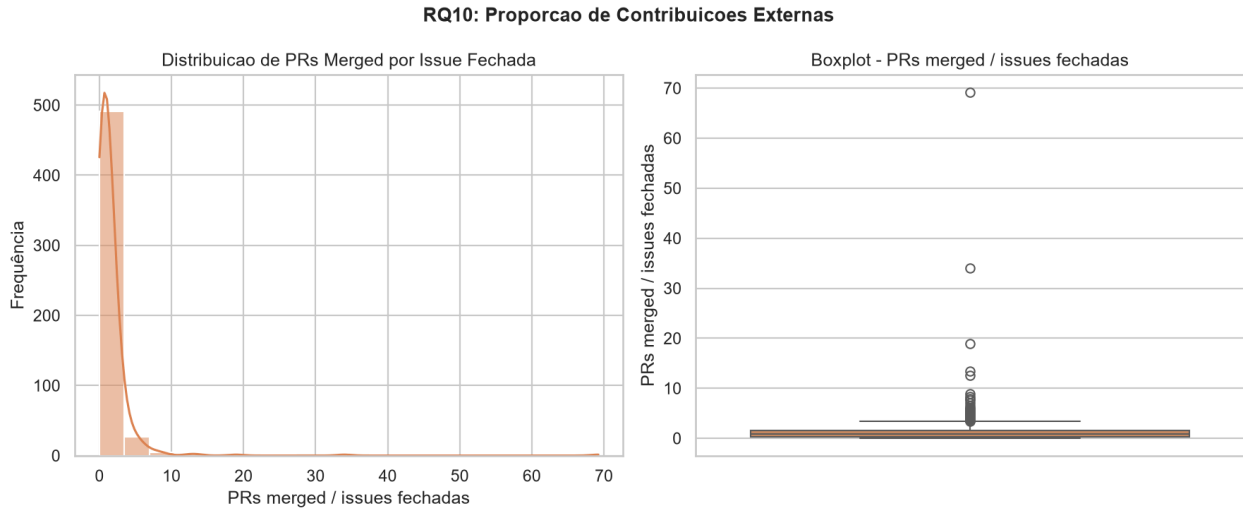
(Mediana: 0,5000 PRs mergeadas por issue fechada)

Pesquisa:



(Mediana: 0,9064 PRs mergeadas por issue fechada)

Geral:



(Mediana: 0,8991 PRs mergeadas por issue fechada)

## 4.3 Discussão

RQ01 (Idade). A hipótese de que sistemas populares são maduros/antigos se confirma nos três domínios, com medianas muito próximas entre si (Game Dev: 7,85 anos; Geral: 7,82 anos; Ciência/ML: 7,75 anos) — a idade elevada e consistente entre domínios sugere que popularidade (mais de 1.000 estrelas) é, em geral, um atributo que leva anos para se acumular, independente da área de aplicação.

RQ02 (PRs aceitas). Aqui a hipótese de alta contribuição externa se confirma de forma desigual: o domínio Geral tem mediana de 878 PRs aceitas, muito acima de Ciência/ML (124) e Game Dev (53). A diferença de mais de uma ordem de grandeza sugere que repositórios de infraestrutura/ferramentas de propósito geral atraem consideravelmente mais colaboração externa do que projetos de nicho técnico (jogos, pesquisa em IA), possivelmente por terem base de usuários-desenvolvedores maior e barreira de entrada técnica menor.

RQ03 (Releases). Mesmo padrão do RQ02: mediana de 36 releases no domínio Geral contra apenas 3–5 nos outros dois, reforçando a hipótese de que a frequência de releases acompanha o padrão de contribuição externa — projetos com processo de release mais maduro tendem a ser os mesmos que recebem mais PRs.

RQ04 (Última atualização). Resultado contraintuitivo: o domínio Geral tem mediana de apenas 2 dias desde a última atualização (extremamente ativo), enquanto Game Dev (55 dias) e, principalmente, Ciência/ML (154 dias) são bem menos recentes. A hipótese informal de que "todo projeto popular é atualizado com frequência" só se confirma fortemente no domínio Geral; para Ciência/ML, é plausível que muitos repositórios sejam associados a publicações acadêmicas específicas (código de um paper), que recebem pouca manutenção após a divulgação inicial — uma hipótese que merece investigação



futura.

RQ05 (Linguagens). O perfil de linguagens reflete fortemente o domínio: Game Dev é dominado por C++, Python, C# e GDScript (linguagens de engines como Godot/Unity/Unreal); o domínio Geral é dominado por Python, TypeScript e JavaScript (ecossistema web/ferramentas); Ciência/ML é dominado esmagadoramente por Python e Jupyter Notebook (611 dos 1.000 repositórios, mais de 60%), consistente com o ecossistema consolidado de bibliotecas científicas em Python.

RQ06 (Taxa de resolução de issues). Ver seção 4.2 — hipótese confirmada nos três domínios (todos acima de ~69%), com o domínio Geral discretamente à frente.

RQ08 (Inovação — issues por estrela). Esta métrica revela uma dimensão que o RQ06 sozinho não captura: apesar de fechar issues numa taxa similar aos demais, o domínio Game Dev gera quase o dobro de issues por estrela de popularidade, sugerindo maior "atrito" relativo — possivelmente porque jogos/ferramentas de arte lidam com grande diversidade de hardware/plataformas, gerando mais bugs reportados por usuário engajado.

RQ09 (Inovação — atividade continuada). Em todos os domínios, o tempo médio entre criação e último push ultrapassa 2.200 dias (mais de 6 anos), com Geral (2.716 dias, mediana) e Game Dev (2.375 dias) à frente de Ciência/ML (2.231 dias). Isso complementa o RQ01 e o RQ04: não basta um repositório ser antigo (RQ01) — ele também segue recebendo commits por praticamente toda a sua existência, refutando a hipótese de que repositórios populares tendem a ser "abandonados" após o pico inicial de popularidade, embora Ciência/ML mostre o padrão relativamente mais fraco dos três, coerente com a última atualização mais distante observada no RQ04.

RQ10 (Inovação — Proporção de contribuições externas). A mediana foi de 0,5000 em Arte, 0,8991 no Geral e 0,9064 em Ciência/ML, indicando maior proporção de PRs mergeadas nos dois últimos grupos. A RQ10 complementa a RQ02 ao relacionar contribuições de código às issues fechadas, funcionando como um indicador aproximado de abertura à colaboração externa.

Ameaças à validade. (i) A segmentação por domínio via palavras-chave de busca pode excluir repositórios relevantes que não usam esses termos exatos nos seus tópicos/descrição (viés de seleção); (ii) não há confirmação de que o mesmo filtro stars:>1000 foi aplicado de forma idêntica nas três buscas; (iii) a categoria "Não informada" em RQ05 mistura causas distintas (fork sem linguagem detectável, repositório arquivado, limitação do GitHub Linguist) sem diferenciá-las; (iv) a coleta é um snapshot único — não captura variação ao longo do tempo; (v) a diferença considerável entre média e mediana em quase todas as métricas (ex.: PRs aceitas no domínio Geral: média 5.098 vs. mediana 878) confirma distribuição fortemente assimétrica com outliers de grande influência, reforçando que a mediana — e não a média — deve ser a estatística central reportada nas conclusões.

## 5. Conclusão

Os resultados confirmam, de forma consistente entre os três domínios analisados, que repositórios populares no GitHub tendem a ser maduros (média de 7 a 8 anos de idade) e a manter atividade de

desenvolvimento por praticamente toda a sua existência (RQ09), refutando a ideia de que popularidade é seguida de abandono rápido. A taxa de resolução de issues também se mostra consistentemente alta (69% a 76%) independentemente do domínio, sugerindo que projetos populares mantêm, em geral, um fluxo de triagem funcional.

Por outro lado, o volume de contribuição externa (PRs aceitas, releases) e a frequência de atualização variam de forma acentuada por domínio — repositórios de propósito geral recebem ordens de grandeza mais PRs e releases, e são atualizados com muito mais frequência, do que projetos de nicho (jogos/arte, ciência/ML). A métrica adicional proposta pelo grupo (RQ08 — issues por estrela) revelou que o domínio de Game Dev/Arte gera proporcionalmente mais atrito de suporte por estrela do que os demais, uma informação que passaria despercebida olhando apenas para a taxa de fechamento (RQ06).

A RQ10 também complementou a análise da contribuição externa ao relacionar PRs mergeadas com issues fechadas. A mediana foi de 0,5000 em Game Dev/Arte, 0,8991 no domínio Geral e 0,9064 em Ciência/ML/IA, indicando uma proporção relativa maior de PRs incorporadas nos dois últimos grupos. O resultado reforça que o volume absoluto de PRs observado na RQ02 pode ser complementado por uma medida relativa ao volume de demandas encerradas. Entretanto, como a métrica não diferencia PRs internas de contribuições efetivamente externas, seus resultados devem ser interpretados como um indicador aproximado de abertura à colaboração por código.

Limitações do estudo: a amostra foi coletada como três subamostras temáticas em vez de um único top-1.000 geral, o que diverge do enunciado original e limita comparações diretas com estudos que usam amostra não segmentada; a coleta é pontual (um único snapshot), sem verificação de tendência ao longo do tempo; e nenhuma das diferenças observadas entre domínios foi validada com teste estatístico (as comparações aqui são descritivas, não inferenciais).

O que o grupo faria diferente com mais tempo: coletar também uma amostra única sem filtro temático (stars:>1000) para servir de linha de base literal ao enunciado; aplicar um teste não paramétrico (ex.: Kruskal-Wallis) para confirmar se as diferenças entre domínios observadas em RQ02, RQ04 e RQ08 são estatisticamente significativas ou apenas variação amostral; e verificar/remover eventuais repositórios duplicados entre as três buscas temáticas.

Inovações que valeriam expansão futura: o RQ08 (issues por estrela) se mostrou o indicador mais discriminante entre domínios nesta análise e merece ser cruzado com outras variáveis (ex.: idade, linguagem) em trabalhos futuros; a segmentação por domínio, se formalizada como inovação intencional (e não um desvio não declarado do enunciado), tem potencial para se tornar um eixo comparativo permanente nos próximos laboratórios do semestre.

## 5. Referências

- ALRASHEDY, Kamel; BINJAHLAN, Ahmed. How do software engineering researchers use github? an empirical study of artifacts & impact. In: **2024 IEEE International Conference on Source Code Analysis and Manipulation (SCAM)**. IEEE, 2024. p. 118-130.